PERGAMON

Computers & Operations Research 27 (2000) 183–203

computers & operations research

www.elsevier.com/locate/orms

Heuristics for the Mixed Rural Postman Problem

Angel Corberán*, Rafael Martí, Antonio Romero

Departamento de Estadística e Investigación Operativa, Facultad de Matemáticas, Universidad de Valencia, Spain

Received 1 November 1997; received in revised form 1 October 1998

Abstract

The Rural Postman Problem on a mixed graph (MRPP) consists of finding a minimum cost tour which traverses, at least once, the arcs and edges of a given subset of the arcs and edges of the graph. This problem is known to be NP-hard. This paper presents two heuristic approaches to solve it. An approximate algorithm based on the resolution of some flow and matching problems and a tabu search implementation is presented. The tabu search algorithm seeks high-quality tours by means of a switching mechanism in an intensification phase and two levels of diversification. Computational results are presented to assess the merits of the method.

Scope and purpose

Routing Problems arise in several areas of distribution management and logistics and their practical significance has been well documented in the literature. Arc Routing Problems are routing problems where a single vehicle or a fleet of vehicles must service the arcs of a transportation network. Such problems occur in a variety of practical contexts, such as garbage collection, mail delivery, snow removal and school bus routing. In this paper we focus on the Mixed Rural Postman Problem (MRPP), an NP-hard problem that includes, as special cases, the well-known Chinese Postman Problem and the Rural Postman Problem. The MRPP fits real-world situations more closely than other simpler problems. In this paper we present heuristic procedures for the MRPP, where the effectiveness of the procedures is measured by the combination of solution quality and speed. © 2000 Elsevier Science Ltd. All rights reserved.

Keywords: Arc routing; Postman problems; Tabu search; Metaheuristics

* Corresponding author.
E-mail address: angel.corberan@uv.es (A. Corberán)

0305-0548/00/$ - see front matter © 2000 Elsevier Science Ltd. All rights reserved.
PII: S0305-0548(99)00031-3

1. Introduction

Given a connected undirected graph or multigraph $G = (V, E)$, with a set of vertices V, a set of edges E and a nonnegative cost vector associated with E, the celebrated Chinese Postman Problem (CPP) is that of finding a minimum-cost closed walk traversing each edge of E at least once. This problem, as well as its directed version, the CPP defined on a directed graph (DCPP), is solvable in polynomial time [1], whereas the corresponding problem defined on a mixed graph is known to be NP-hard [2]. In the Mixed Chinese Postman Problem (MCPP) the edges of the graph (two way streets) may be traversed in both directions with the same cost, while the arcs (one way streets) have to be traversed in only one specified direction. This interesting extension of the CPP reflects real-life situations more directly. Heuristic algorithms for the MCPP have been proposed by Frederickson [3] and Pearn and Liu [4]. Grötschel and Win [5] and Nobert and Picard [6] devised LP-based cutting-plane algorithms solving to optimality instances of medium sizes.

On the other hand, another more practical generalization of the CPP is the Rural Postman Problem (RPP), in which only a subset, E_R, of the edges in E are required to be traversed at minimum cost. The RPP and its directed version, the Directed Rural Postman Problem (DRPP), are both NP-hard [7]. Recent heuristic procedures for the RPP have been proposed by Pearn and Wu [8] and Hertz et al. [9], while its associated polyhedron is studied in Corberán and Sanchis [10,11] and Letchford [12].

All the above-mentioned problems are included in the class of Arc Routing Problems, those routing problems where a vehicle or a fleet of vehicles must service the "arcs" of a transportation network for the purpose of garbage collection, mail delivery, snow removal, inspection of electric power lines or oil or gas pipelines, street sweepers, meter reading, spraying roads with salt and school bus routing, and have been the object of great attention by operations researchers. Excellent surveys on the area are those recently published by Eiselt et al. [13,14] and Assad and Golden [15].

In this paper we focus on the RPP defined on a mixed graph, the Mixed Rural Postman Problem (MRPP), a problem including as special cases all those mentioned above, and that accommodates real-world situations more closely. In fact, some of us faced this problem while designing a code for the garbage collection in medium size cities [16].

The MRPP may be defined as follows. Let $G = (V, E, A)$ be a strongly connected mixed graph, where $V = \{1, 2, \ldots, n\}$ is the vertex set and E and A are the set of edges and arcs, respectively. The arcs or edges of G will be termed the links of the graph. A nonnegative cost c_{ij} is associated with every link (i, j). Let $E_R \subseteq E$ and $A_R \subseteq A$ be the set of (required) edges and (required) arcs, respectively, that must be serviced. Then, the MRPP is to find a minimum cost closed walk (tour) traversing each required link (edge or arc) at least once. Note that if (i, j) is an edge of cost c_e, then the cost of traversing (i, j) in either direction is c_e, while if (i, j) is an arc of cost c_e, then the cost of traversing (i, j) is c_e if the route goes from i to j, and ∞ otherwise. Obviously, the MCPP can be derived from the MRPP for $E_R = E$ and $A_R = A$, while the RPP is derived from the MRPP for $A = \emptyset$. Then, the MRPP is a generalization of two NP-hard problems and, hence, is itself NP-hard.

In this paper we present two heuristic algorithms for the MRPP: a constructive algorithm to obtain good solutions in short computation time (Section 3), and a local search procedure that seeks high-quality solutions (Section 5). The first one is partially based on the algorithm for the MCPP proposed in [17]. The second one uses different tabu search elements, such as memory structures, attractiveness based on influence and probabilistic strategies to overcome local

optimality. In Section 2 some simple graph transformations and notation are presented. Finally, Section 6 describes the lower bound we have computed to assess the merits of the heuristics as well as the computational experiments on a set of 270 randomly generated instances.

2. Graph transformations and notation

As is usual in Rural Postman Problems, in order to simplify the resolution of the MRPP we first reduce the problem by some simple graph transformations. Let $G_R = (V_R, E_R, A_R)$ be the graph induced by $E_R \cup A_R$, where V_R is the subset of vertices in V which are incident to at least a link of $E_R \cup A_R$. From G_R we construct a new graph $G' = (V_R, E_R, A_R \cup A_{NR})$, where A_{NR} is obtained by first adding an arc (i, j) for each pair of vertices $i, j \in V_R$ with cost equal to the length of a shortest path from i to j (calculated in the original graph G); then deleting all arcs $(i, j) \in A_{NR}$ for which $c_{ij} = c_{ik} + c_{kj}$ for some $k \in V_R$, as well as any arc in A_{NR} parallel to an edge in E_R having the same cost, or to an arc in A_R with the same cost and direction. (See [14] or [18] for an illustration of the procedure.)

In some cases it is possible to further simplify graph G' in three ways: replacing required edges with arcs, reducing the number of connected components induced by the set $E_R \cup A_R$ in G' and computing the minimum number of times some arcs have to be traversed. The procedures to do this are very simple and are based on similar procedures described in Christofides et al. [17] for the MCPP and in [18] for the DRPP.

It is easy to see that solving the MRPP over the original graph is equivalent to solving it on the resulting graph after having applied the above transformations, that, for notational convenience, will still be denoted by $G = (V, E, A)$, $A = A_R \cup A_{NR}$. Note that all edges in E are required. The set of required links $E \cup A_R$ induces p connected components in G, which will be called R-connected components, $G_1, G_2, \ldots, G_p$, with vertex sets $V_1, V_2, \ldots, V_p$, respectively. Arcs $(i, j) \in A_{NR}$ such that $i \in V_k$, $j \in V_l$, $k \neq l$ will be called connecting arcs and A_C will denote this arc subset. Given $S \subset V$, $A^+(S)(A^-(S))$ will represent the set of arcs leaving (entering) S and $E(S)$ will denote the set of edges between S and $\bar{S} = V \backslash S$. Furthermore, $d^-(i)(d^+(i))$ is the indegree (outdegree) of vertex i with respect to the required arcs, i.e. the number of required arcs entering (leaving) vertex i.

A solution (tour) for the MRPP can be characterized [19] as any family T of links such that the graph induced by T, $G(T)$, is strongly connected, contains $E \cup A_R$, is even (every vertex is incident with an even number of links in T) and is balanced, i.e. $\forall S \subset V$, $|A^+(S)| - |A^-(S)| \leqslant |E(S)|$ (balanced sets condition). Note that if $E(S) = \emptyset$, the two balanced set conditions for S and $\bar{S}$ give the so called symmetry condition $A^+(S) = A^-(S)$ for set S.

3. A constructive procedure

Let $G = (V, E, A)$ be the strongly connected mixed graph obtained from the original graph after having applied the previous transformations. Remember that all edges in E are required, all vertices in V are incident with required links and $A = A_R \cup A_{NR}$. In this section we present a heuristic algorithm for the MRPP consisting of 4 stages:

Stage 1: *Connectivity.* Since the tour must connect $G_1, G_2, \ldots, G_p$, our algorithm looks for a subset $A' \subseteq A_C$ of connecting arcs such that the graph induced in G by $E \cup A_R \cup A'$ will be

connected. This is accomplished by first calculating new costs associated to arcs in A_C and, then, computing a shortest spanning tree connecting $G_1, \ldots, G_p$. For each $(i, j) \in A_C$, its original cost c_{ij} is modified in accordance with the values of $d^+(i)$, $d^-(i)$, $d^+(j)$ and $d^-(j)$. The idea is to look for a connected graph that would be produced, in a second phase, balanced at a low extra cost. The procedure is as follows:

(a) For each $i \in V$, let $D(i) = d^-(i) - d^+(i)$ be the supply (if positive) or demand (if negative), with respect to the required arcs, of vertex i. Compute

$$c_i^+ = \min\{c_{ik}: (i, k) \in E \cup A\} \quad \text{(cost of the shortest link leaving } i\text{)},$$

$$c_i^- = \min\{c_{ki}: (k, i) \in E \cup A\} \quad \text{(cost of the shortest link entering } i\text{)},$$

set $c'_{ij} := c_{ij}$, $\forall (i, j) \in A_C$ and $D'(i) := D(i)$, $\forall i \in V$.

(b) Suppose arcs in A_C have been ordered accordance to their increasing costs. Starting with the least cost arc $(i, j) \in A_C$:

(i) Assign a "direction" to all edges (if any) incident with i and j, depending on the values of $D'(i)$ and $D'(j)$. For example, given $(k, i) \in E$, if $D'(k) \geqslant D'(i)$, then $D'(k) := D'(k) - 1$ and $D'(i) := D'(i) + 1$ (edge (k, i) will be considered as "directed" from k to i); otherwise, $D'(k) := D'(k) + 1$ and $D'(i) := D'(i) - 1$.

(ii) Modify cost c'_{ij} in the following way: If $D'(i) \leqslant 0$, a penalty for using arc (i, j) is paid: $c'_{ij} := c'_{ij} + c_i^-$; otherwise $(D'(i) > 0)$, a benefit is obtained: $c'_{ij} := c'_{ij} - c_i^+$. Moreover, if $D'(j) \geqslant 0$, $c'_{ij} := c'_{ij} + c_j^+$; otherwise, $c'_{ij} := c'_{ij} - c_j^-$.

Repeat steps (i) and (ii) for all arcs in A_C.

(c) With respect to costs c'_{ij}, find a shortest spanning tree A' connecting $G_1, G_2, \ldots, G_p$. Each time an arc $(i, j) \in A_C$ is selected to be in the tree, modify $D'(i)$ and $D'(j)$.

Stage 2: *Improving connectivity*. Assign a "direction", as in step $b(i)$, to all edges remaining "undirected" and update the supplies and demands of their terminal vertices. Then, as in the heuristic algorithm proposed in [17] for the MCPP:

(a) Construct a directed graph $G_F = (V, A \cup A_F)$ as follows: for each edge $(i, j) \in E$, add two arcs in parallel, one in each direction, in A_F, with costs equal to that of the edge. Moreover, add an "artificial" arc of zero cost in the opposite direction to the one previously assigned to the edge. Arcs of A appear with original costs c_{ij}.

(b) Solve in G_F a minimum cost flow problem, in which a vertex i with $D'(i) > 0$ (< 0) is a source (sink) with supply (demand) $D'(i)$($-D'(i)$) and the capacity of each artificial arc is 2 and ∞ that of the remaining arcs.

As will be shown in Stage 3, from the solution to this flow problem we obtain a feasible solution to the MRPP traversing, at least once, all arcs in $A' \subseteq A_C$. The objective of this stage is to try to improve the way in which $G_1, \ldots, G_p$ have been connected, once we know what kind of solution we would obtain from it. The idea is very simple and is described in what follows:

Let $A'' = \emptyset$. For each arc $(i, j) \in A'$, $i \in V_k$, $j \in V_l$, if in the solution to the above flow problem:

- there are no arcs between G_k and G_l, then $A'' := A'' \cup (i, j)$;
- there are some arcs in the same direction as (i, j) (i.e. from G_k to G_l), then include in A'' the minimum cost arc among them;

– all arcs between G_k and G_l are in the opposite direction to (i, j), then include in A'' the minimum cost arc among them, except if there is only one arc (s, t), $s \in V_l$, $t \in V_k$; in such a case, do not add any arc to A'' but label arc (s, t) as "studied".

If the graph induced by $E \cup A_R \cup A''$ is not connected, we look for a shortest spanning tree linking the connected components of this graph with arcs in the solution to the flow problem (that are added to A''), using first, if possible, non-labeled arcs. Finally, compute the new supplies and demands, $D''(i)$, with respect to the arcs in $A_R \cup A''$.

Stage 3: *Obtaining a feasible solution*. On the graph G_F of Stage 2, solve again a minimum cost flow problem with demands and supplies given now by $D''(i)$.

3.1. Constructing the balanced mixed graph $G_B = (V, E_B, A_B)$

Let $f(i, j)$ and $f^a(i, j)$ represent the number of flow units circulating in the optimal solution through non-artificial and artificial arcs, respectively. From the resolution of the flow problem, construct graph G_B as follows:

– if $f^a(i, j) = 0$, include $1 + f(j, i)$ arcs (j, i) in A_B,
– if $f^a(i, j) = 2$, include $1 + f(i, j)$ arcs (i, j) in A_B,
– if $f^a(i, j) = 1$, include edge (i, j) in E_B,
– $\forall (i, j) \in A_R \cup A''$, include $1 + f(i, j)$ arcs (i, j) in A_B,
– $\forall (i, j) \in A_{NR} \setminus A''$, include $f(i, j)$ arcs (i, j) in A_B.

It is easy to see that G_B is a strongly connected graph containing all the required links at least once. Moreover, since the number of arcs entering and leaving each vertex is equal, G_B is a balanced graph. If, in addition, $E_B = \emptyset$, then G_B is also an even graph, representing a feasible solution to the MRPP on graph G. Otherwise, if $E_B \neq \emptyset$, some vertices incident with edges in E_B may have odd degree and then we proceed to step 3.2 in order to transform G_B into an even graph.

3.2. Converting G_B into an even graph

Identify the odd degree vertices in the subgraph induced by E_B on G_B. Let V_M be such a set. Construct an undirected and complete graph with set of vertices V_M and the following costs associated to their edges: $c_{ij} = \min\{s(i, j), s(j, i)\}$, where $s(i, j)$ represents the cost of the shortest path from i to j on graph G. Solve a minimum cost matching problem on this graph and let E_M be the set of edges in the matching. Let A_M be the set of shortest directed paths associated to the edges in E_M. If, in the graph (V_M, E_B, A_M), an orientation of the edges in E_B to obtain a directed and symmetric graph (although it may be disconnected) is possible, it is achieved: adding these cycles to $G_B \setminus E_B$, we obtain an even and balanced graph that represents a feasible solution for the MRPP. If, however, this orientation of edges in E_B is not possible, as in the example shown in Fig. 1, one (or more) of the directed paths is changed by the path (or paths) in the opposite direction. Using the example in Fig. 1, if $s(i_1, i_2) + s(j_2, j_4) + s(i_4, j_1) \leqslant s(j_1, i_4) + s(j_4, j_2) + s(i_2, i_1)$, substitute in A_M paths from j_1 to i_4 and j_4 to j_2 by their opposite ones. Otherwise, change the path from i_1 to i_2 to the reverse one.

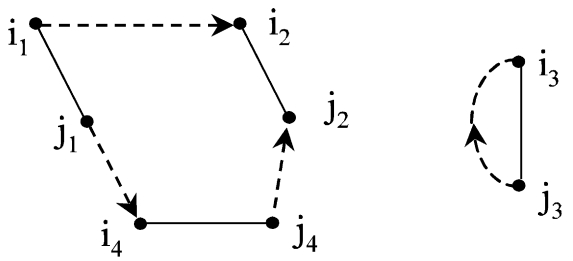


Fig. 1

Stage 4: *Improvement of the solution*. At this point of the algorithm, a feasible solution for the MRPP, represented by a completely directed graph $G_d = (V, A_d)$, has been obtained. Note that a direction has been assigned to all edges in E, from the solution to the flow problem defined in Stage 3 and, if it is the case, from the directed cycles that, added to $G_B \backslash E_B$, made this graph even. Let us represent by E^d the set of "directed" edges. Arcs in $A_d \backslash \{A_R \cup E^d\}$ are an augmentation of the graph induced by the required links, i.e. a set of arcs that will make this graph Eulerian.

A simple, but useful, procedure for improving a feasible solution in the case of the MCPP follows. If the cost c_{ij} of an arc, or a path, in the augmentation of the graph exceeds the length of a shortest path from i to j, it is advantageous to replace the arc, or path, by the shortest path. The objection to this procedure is that, for example, depending on the order in which arcs are examined, it can lead to different improvements. Furthermore, it cannot be directly applied to the MRPP, as changing one arc (or path) by a shortest path may produce a disconnected graph.

The procedure we suggest here circumvents the last difficulty and attempts to improve the solution in a global way. In the graph induced by $A_d \backslash \{A_R \cup E^d \cup A''\}$ compute the supplies and demands of their vertices and solve a minimum cost flow problem with these values over the graph G. By adding the solution to the above flow problem to the graph $(V, A_R \cup E^d \cup A'')$, a feasible solution of less or equal cost is obtained.

4. Illustration of the constructive procedure

Figs. 2 and 3 show the original graph and the simplified graph on which the constructive procedure will be applied. Required links are represented by solid lines and define four R-connected components whose vertex sets are given by $V_1 = \{1, 4, 5, 9\}$, $V_2 = \{2, 3, 6, 7, 8, 13, 14, 15\}$, $V_3 = \{10, 11, 12, 16, 17, 21, 22, 23\}$ and $V_4 = \{18, 19, 20, 24, 25\}$.

Stage 1: *Connectivity*. Initially, supplies and demands $D(i)$ are computed with respect to the required arcs. For example, $D(1) = -1$ and $D(2) = +1$.

Starting with the least cost connecting arc, (19, 14) (or (14, 19) that has the same cost), a direction is given to all edges incident with vertices 19 and 14: edge (19, 20) is directed from 20 to 19 since $D(19) = -1$ and $D(20) = +1$. Now $D'(19) = D(19) + 1 = 0$ and $D'(20) = D(20) - 1 = 0$. Similarly, edge (19, 24) is directed from 24 to 19 and edge (7, 14) is directed from 7 to 14.

In Fig. 4, small positive numbers located near a vertex i incident with arcs in A_C correspond to c_i^+ and small negative numbers to $-c_i^-$. For example, $c_1^+ = 32$ and $c_1^- = 35$. Also, Fig. 4 shows in dashed lines all arcs in A_C with their modified costs c'_{ij}.

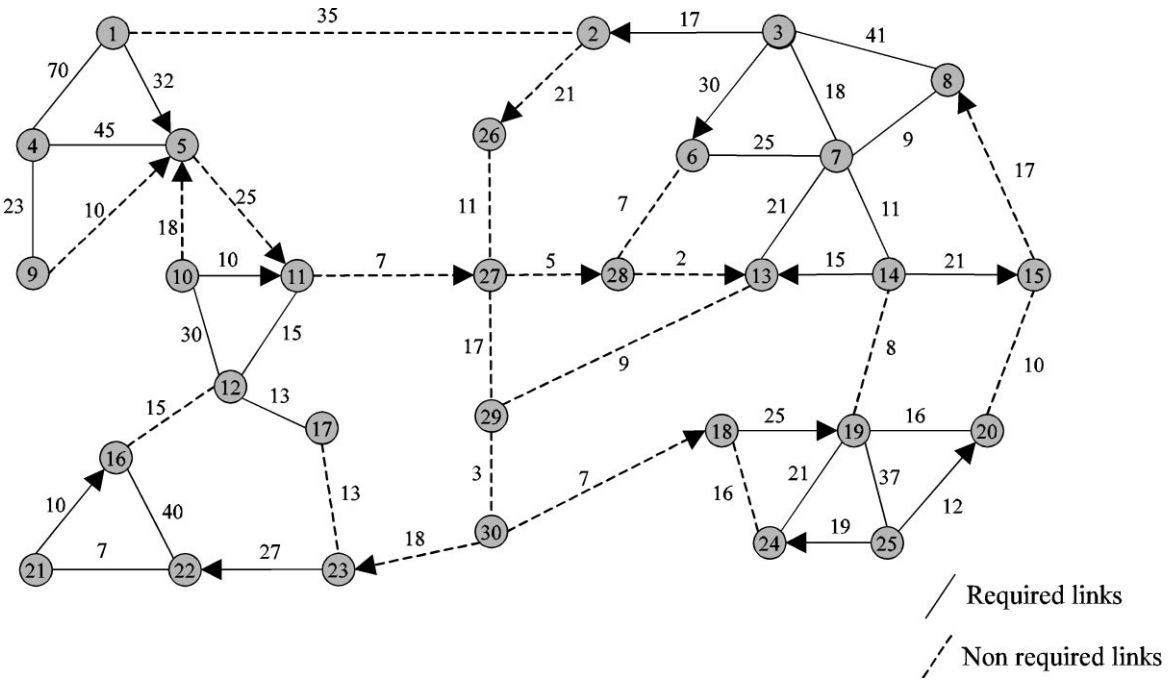


Fig. 2. Original mixed graph.

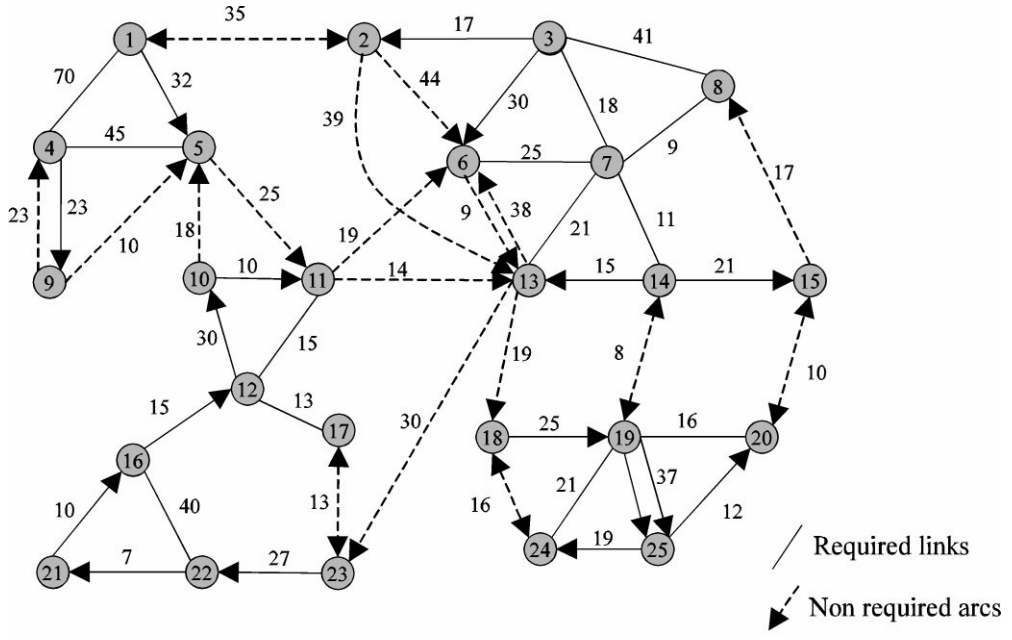


Fig. 3. Simplified mixed graph.

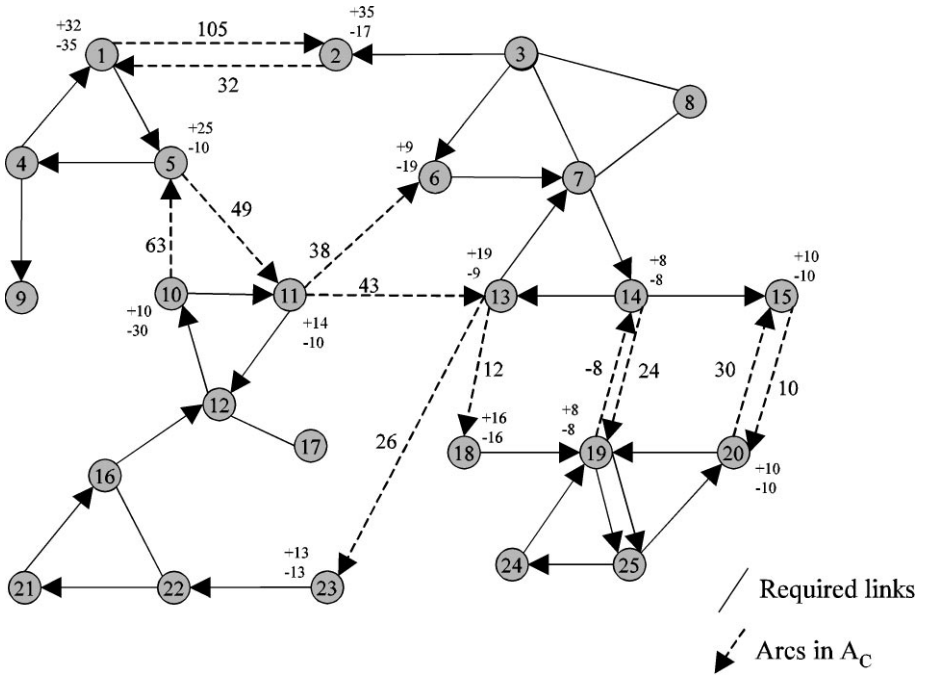


Fig. 4

With respect to distances in Fig. 4, a shortest spanning tree connecting the four R-connected components is computed. It is given by the set of arcs $A' = \{(19, 14), (13, 23), (2, 1)\}$ represented by an arrow with double line in Fig. 5.

Stage 2: *Improving connectivity.* First, a direction to the remaining "undirected" edges is assigned. Then, supplies and demands are computed with respect to the required links and the arcs in A'. Note that in this step all edges have a direction as if they were arcs. Now, a minimum cost flow problem is solved in a modified graph G_F. From the solution to this flow problem, we obtain a connected and balanced graph that has been drawn in Fig. 5.

The procedure tries to get another set of connecting arcs A'' as follows. Consider, first, arc $(19, 14) \in A'$, $19 \in V_4$ and $14 \in V_2$. As in the flow problem solution there is only one arc (13, 18) between G_4 and G_2 and it has the opposite direction, so arc (13, 18) is labeled as studied. The procedure follows with arc (13, 23), $13 \in V_2$ and $23 \in V_3$. Since in the flow solution there are two copies of arc (11, 13) with the opposite direction, we include arc (11, 13) in A''. Finally, as there are no arcs in the solution to the flow problem between G_1 and G_2, arc (2, 1) is included in A''.

The graph induced by $A_R \cup E \cup A''$ is not connected. Its connected components are linked using the flow arc (13, 18), that is added to A''. Supplies and demands computed with respect to $A_R \cup A''$ are represented by $D''(i)$. Arcs in A'' are represented by double line arrows in Fig. 6.

Stage 3: *Obtaining a feasible solution.* Solving a minimum cost flow problem with demands and supplies $D''(i)$, we get a balanced mixed graph that is shown in Fig. 6. This graph has some

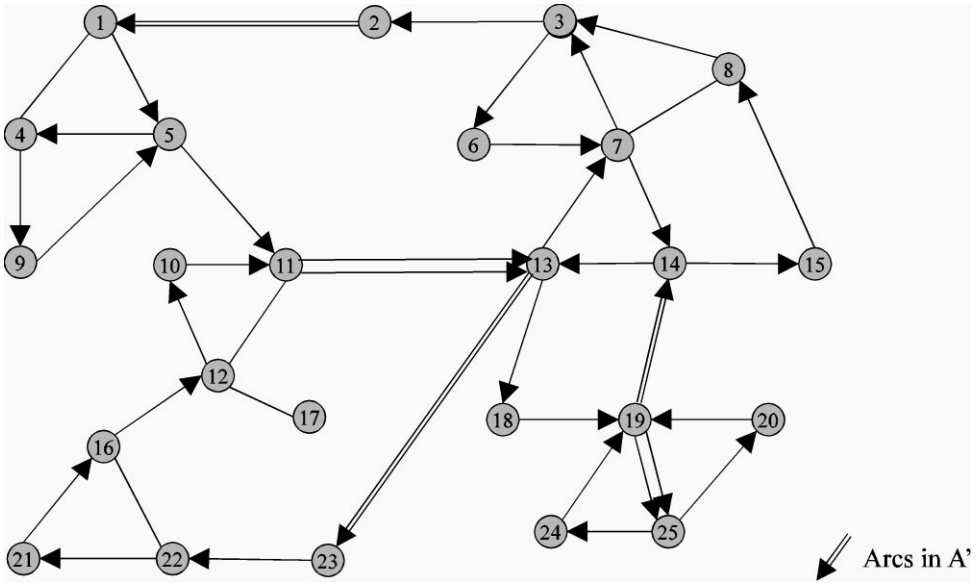


Fig. 5. Balanced mixed graph obtained from the initial connection.

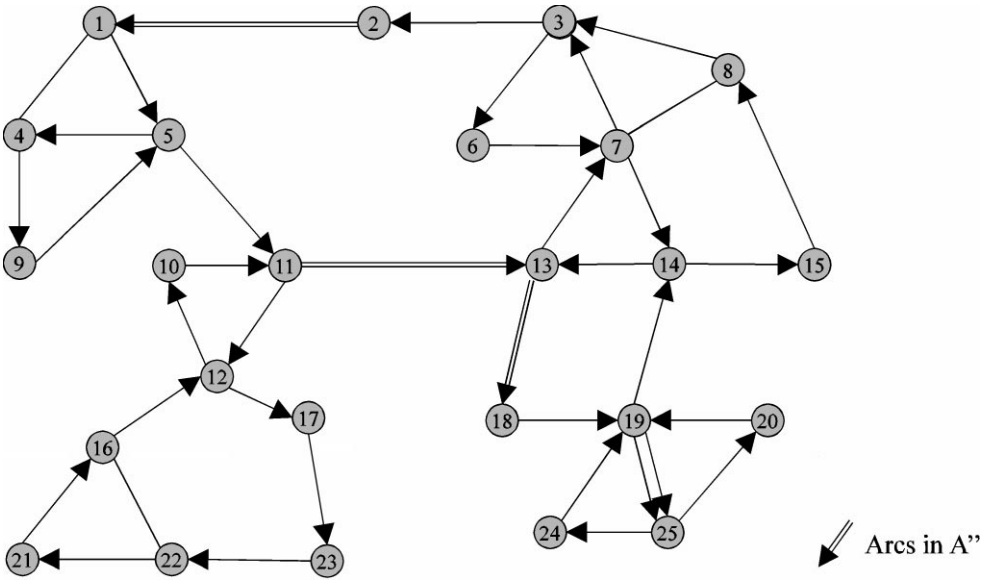


Fig. 6. Balanced mixed graph of Stage 3.1.

odd-degree vertices: 1, 4, 7, 8, 16 and 22. Solving a minimum cost-matching problem as described in step 3.2, we obtain a feasible MRPP solution, represented by a completely directed graph, depicted in Fig. 7.

Stage 4: *Improvement of the solution.* Deleting arcs not in $A_R \cup E^d \cup A''$ from the graph in Fig. 7, we compute supplies and demands shown in Fig. 8. Solving a minimum cost flow problem with these values, the MRPP solution with cost 924 is obtained (Fig. 8).

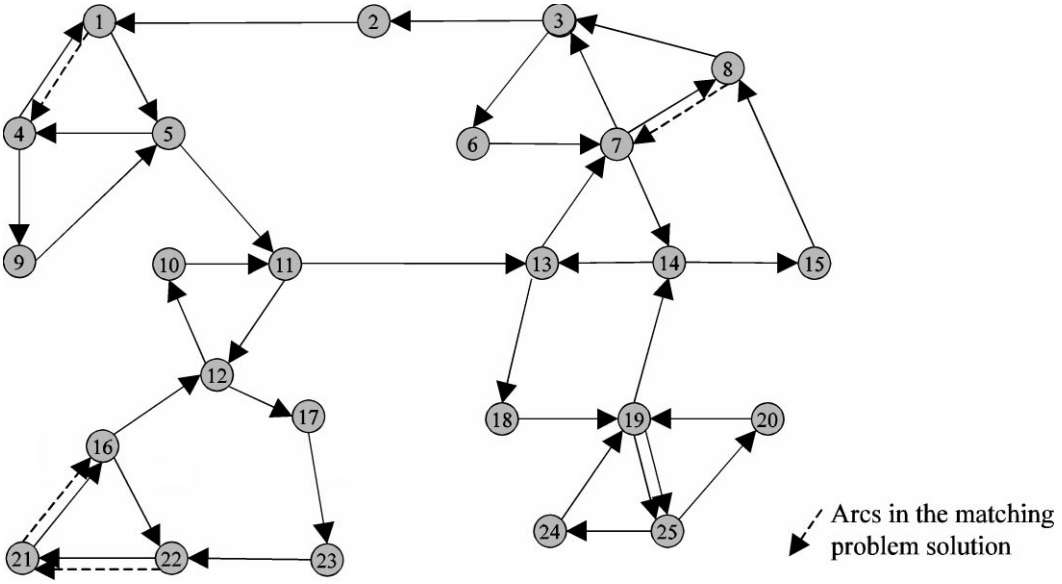


Fig. 7. MRPP solution with cost 949.

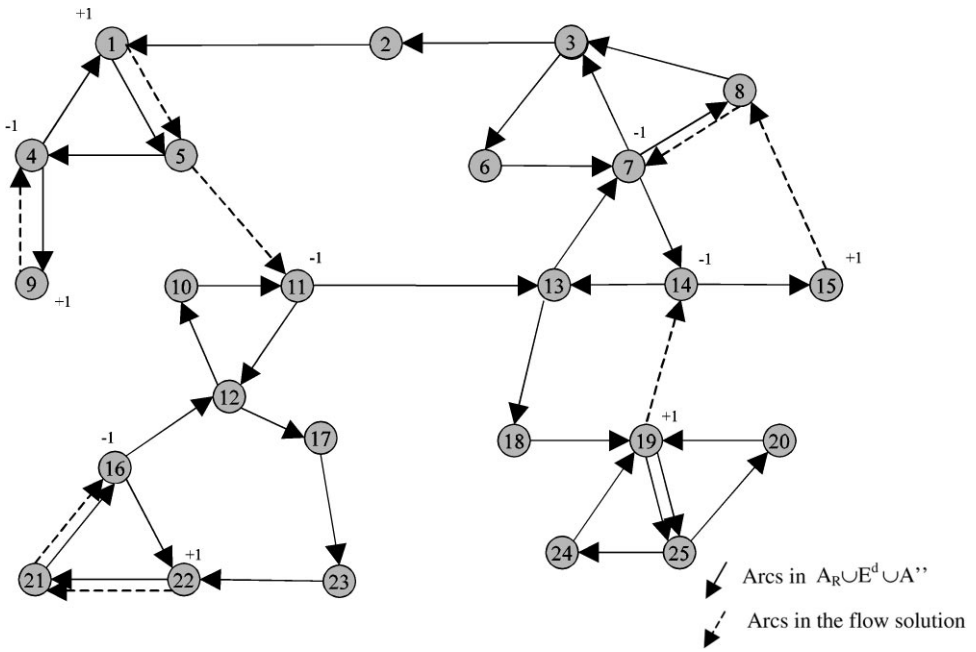


Fig. 8. Improved solution with cost 924.

5. Tabu search algorithm

Tabu Search (TS) is an adaptive technique for solving hard combinatorial optimization problems based on general principles of intelligent problem solving. In essence it is a metaheuristic, which may be used to guide any local search procedure in the aggressive search for a global optimum, using memory structures to avoid getting trapped in local optima. A complete and precise description of TS may be found in [20]. The following subsections describe the elements that are critical in the development of the tabu search procedure for the Mixed Rural Postman Problem.

Our procedure starts from a MRPP tour obtained with the constructive algorithm. It is represented by a completely directed and strongly connected graph, containing all the required links (a direction has been assigned to original edges) and satisfying the symmetry condition (the number of arcs entering and leaving each vertex is the same). As during the search process disconnected "tours" (i.e., symmetric graphs consisting of several subtours) will be allowed, we will also use the term solution to designate them, for the sake of simplicity.

5.1. Neighborhood definition

Switches are used as the primary mechanism in our TS scheme to move from one solution to another. Let (i, j) be a connecting arc and let us denote by $[k, i]$ and $[j, s]$ paths from k to i and from j to s, respectively, in the augmentation of the graph induced by the required links. We define move $m(i, j)$ as deleting path $[[k, i], (i, j), [j, s]]$ from the current solution, and adding the shortest path from k to s. Fig. 9 illustrates this move.

Each move has an associated value given by the difference between the objective function values before and after the move. It can be computed as the cost of the deleted path $[[k, i], (i, j), [j, s]]$ minus the cost of the added shortest path.

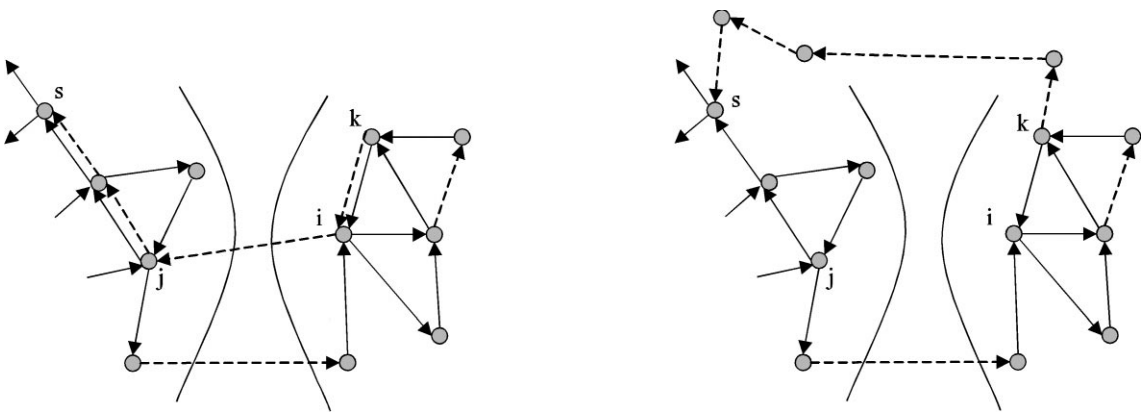


Fig. 9

This operation preserves the conditions for symmetry and evenness, but may result in a disconnected and therefore infeasible solution, since the removed paths might link some R-connected components.

Connecting arc (i, j) could eventually belong to the added shortest path. Given that the goal of this move is to replace one connecting arc for another, in this case we consider move $m(i, j)$ as not available and it would not be performed.

5.2. Basic procedure

Starting from a solution obtained with the constructive algorithm, the basic TS procedure alternates between an intensification and a diversification phase as described below.

5.2.1. Intensification phase

An iteration of this phase begins by randomly selecting a connecting arc. The probability of selecting arc (i, j) is proportional to its cost. Move $m(i, j)$ is executed and the removed arc (i, j) becomes tabu-active for *tabu tenure* iterations, and therefore it cannot be added to the solution during this time.

In subsequent iterations, a connecting arc is randomly selected as before. Its corresponding move is performed if it is available and does not add any tabu arc. Since a shortest path replaces a path from k to s, the objective function decreases (or keeps its value). The intensification phase ends when no connecting arc with an associated available move exists.

5.2.2. Diversification phase

This phase is performed for *maxiter* iterations. In each iteration, a nontabu connecting arc (i, j) is probabilistically selected as in the intensification phase. If move $m(i, j)$ is available and does not add a tabu arc, it is performed. Otherwise, two alternative moves are considered, depending on whether the current solution is connected or not.

If the current solution is connected, the shortest path to be added in the move $m(i, j)$ is computed in the graph obtained by deleting the tabu arcs. Therefore, this restricted move may, or may not, improve the objective function value. The move is executed even when its value is not positive, resulting in a deterioration of the current objective function value.

If the current solution is nonconnected, the algorithm performs the following move in order to connect two components. It randomly selects a connected component and considers all connecting arcs (u, v) from this component in which the shortest paths from v to u do not include a tabu arc. The lowest cost one, considering the arc plus the shortest return path costs, is selected. If all the above paths include a tabu arc, we ignore the tabu status (aspiration criteria by default). This *connecting move* obviously increases the objective function value, but connects two components.

It is worth mentioning that the interplay among these moves keeps a high degree of connectivity in the solution, which is enough to obtain a large number of feasible solutions during the search process.

Both phases are alternated until the global stopping criterion is met. The procedure finishes when, after a number of iterations equal to *nonimprove*, none of the visited solutions improves the best known one.

5.3. Additional diversification (long-term memory)

A long-term diversification phase is implemented to complement the diversification phase in the basic procedure. This phase consists of a re-starting mechanism in which information generated by the search history is used to guide the construction process. The objective is to diversify the search by generating solutions that are different from each other.

In Section 3 we have described an algorithm that first selects a set of connecting arcs and then constructs a solution from this set. Here we apply the same procedure but starting from a set of connecting arcs randomly selected according to a defined value $f(i, j)$. Long-term diversification manages the selection of connecting arcs according to their number of occurrences in previous solutions. These frequencies are mapped onto selection probabilities: the lower the frequency, the higher the probability.

To initialize the process, the costs of connecting arcs are scaled to be merged with frequencies. Let c_{ij} be the cost of connecting arc (i, j) and *cmax* be the maximum of all of them.

$$f(i, j) = \underset{(k,l)\in C}{maximum} \left\lfloor \frac{cmax}{c_{kl}} \right\rfloor - \left\lfloor \frac{cmax}{c_{ij}} \right\rfloor$$

Then, each time arc (i, j) is selected, $f(i, j) = f(i, j) + 1$. The probability of selecting a connecting arc is made inversely proportional to its *f*-value.

We apply the basic procedure to the solutions generated but we only try to improve promising solutions: if the value of the solution is within a percentage of the best solution found, then the basic procedure is applied. Otherwise, this solution is rejected and a new one is generated. Let *Pt* and *Global* be the parameters to manage the percentage deviation from the best solution found and the number of iterations of the procedure, respectively.

Global procedure

---

p = number of R-connected components
iter = 0

While(*iter* < *p*∗*Global*)
  iter = *iter* + 1
  Long Term Diversification (*Generate a solution*)
    Select a connecting arcs set.
    Apply the constructive algorithm.
    Let X be the solution obtained
    Update the frequencies of the connecting arcs.

  If (*Cost*(*X*) < *Pt*∗*Cost*(*Best solution*))
    Apply ***Basic Procedure*** *to X*
    Update, if necessary, Best Solution

---

6. Computational results

The experiments reported in this section were performed on a Pentium-150 PC. All procedures were programmed in C and compiled in Microsoft Visual C++5.0. The instances were generated according to the following parameters: number of R-connected components (p), number of vertices ($|V|$), number of arcs ($|A|$), number of required arcs ($|A_R|$), number of edges ($|E|$) and number of required edges ($|E_R|$).

The graph generator works as follows: vertices are equally distributed among components. Each component has $\lfloor |V|/p \rfloor$ vertices, except possibly the last one which also has the difference up to $|V|$. Arcs and edges are generated, up to a prefixed number, by randomly selecting two vertices and adding the corresponding link.

To generate required links, if the two vertices do not belong to the same component they are rejected and a new selection is made. The final number of nonrequired arcs might be larger, in some instances, than the prefixed number $|A \backslash A_R|$, in order to obtain a strongly connected graph.

This generator allows up to four links between two vertices: one for each kind of link (required and nonrequired arc, required and nonrequired edge). We have randomly generated 100 instances with the following ranges:

$$p \in [3, 12], \quad |V| \in [20, 100], \quad |E| \in [15, 220], \quad |E_R| \in [5, 150], \quad |A| \in [50, 350],$$

$$|A_R| \in [5, 200]$$

The cost of each link has been randomly generated in the range 1–20. These ranges have been chosen in a way similar to that of other arc routing works [8]. Table 7 in the Appendix shows the parameters of each instance.

Preliminary tests, on some of the biggest instances from Table 7 (those with extension "c10"), were performed in order to determine the values of the key parameters in our search scheme. We found that $Pt = 1.3$ and $Global = 10$ produced the best results in terms of solution quality and CPU time. Nevertheless, in the case of *nonimprove* and *maxiter* the results were not so conclusive. Therefore, we have reported seven variants of the TS procedure according to different values of *nonimprove* and *maxiter*. Mixed 1 provides a combination of the tested parameters' values depending on the number p of R-connected components of the instances, as follows: if $p \leqslant 5$, then $nonimprove = 50$ and $maxiter = 50$; if $6 \leqslant p \leqslant 10$, then $nonimprove = 40$ and $maxiter = 75$; finally, if $11 \leqslant p$, then $nonimprove = 25$ and $maxiter = 100$. In order to see the influence of Global, Mixed 2 provides the same combination as Mixed 1 but with Global equal to 15. We have also reported the results of the algorithm described in Section 3, which will be denoted by *Constructive*.

As far as we know, no other heuristic or exact solution method for the MRPP has been published. Then, in order to measure the quality of the solutions obtained by the heuristics described above, we have computed a lower bound to the optimal solution cost. This bound has been obtained with an LP-based cutting plane algorithm from the partial description of the MRPP polyhedron given in [21]. In this study, it is shown that connectivity, R-odd and balanced set inequalities, among others, define facets of the MRPP polyhedron. Therefore, they are among the best cutting planes that can be found. Furthermore, separation problems associated to connectivity and R-odd constraints are solvable in polynomial time by means of max-flow calculations and the Padberg and Rao [22] method to find minimum odd cut-sets. Nobert and Picard [6] show that

identifying violated balanced constraints for the MCPP can also be solved in polynomial time. This result is easily extended to the MRPP.

Our cutting plane procedure starts with a linear program (LP) including the non-negative constraints, connectivity constraints corresponding to the p R-connected components, R-odd inequalities associated to R-odd vertices (those incident with an odd number of required links) and balanced set constraints associated to unbalanced vertices. In the course of the algorithm, additional inequalities of the above-mentioned classes are generated as they are found to be violated. The procedure stops when this is no longer possible. The cost of the optimal solution in the last LP provides a lower bound for the MRPP.

Table 1 reports the average percentage deviation from the lower bound (*LB Dev*) computed as (cost − LB)/LB, the maximum percentage deviation from the lower bound (*MLB Dev*), the average percentage deviation from the best solution found for each instance after running all the Tabu Search variants (*Best Dev*) and the average CPU time in seconds (*Seconds*). *Number of best* provides the number of instances, from the total of 100, in which the procedure is able to find the best solution among all the variants, while *Number of optima* is the number of instances in which the lower bound is reached.

Table 1 shows that all the versions tested, including the constructive procedure, provide high quality results. In general, the average deviations are small for all the Tabu Search variants under consideration, never exceeding more than 0.53% on average. The remarkable performance of the (25, 25) variant must be noted. It achieves an average lower bound deviation of 0.53% and is able to match 27 best solutions in less than 1 min of computer time. The Mixed 2 procedure seems to be superior to the others in terms of solution quality but it is more time consuming.

Table 7 in the Appendix provides the data of the simplified graphs corresponding to the set of 100 instances generated. Table 8 shows, for each instance, the ratio between the solution cost and the lower bound (UB/LB) and the CPU time (*Seconds*) of the constructive procedure. Furthermore,

Table 1
Heuristics performance

Procedure	*LB Dev*	*MLB Dev*	*Best Dev*	*Seconds*	*Number of best*	*Number of optima*
Constructive Procedure	1.87%	20.78%	1.59%	0.5	3	3
nonimprove = 40 *maxiter* = 75	0.49%	3.33%	0.21%	232.1	30	15
nonimprove = 25 *maxiter* = 25	0.53%	7.08%	0.25%	57.8	27	13
nonimprove = 75 *maxiter* = 25	0.53%	5.00%	0.25%	156.1	23	12
nonimprove = 25 *maxiter* = 100	0.50%	6.25%	0.22%	192.4	32	16
nonimprove = 50 *maxiter* = 50	0.48%	3.96%	0.20%	198.0	32	13
Mixed 1	0.50%	6.25%	0.22%	217.7	35	15
Mixed 2	0.47%	6.04%	0.19%	326.3	44	17

Table 2

$\|V\|$	d	Constructive		Tabu	
		UB/LB	*Seconds*	UB/LB	*Seconds*
30	10%	1.0227	0.07	1.0003	55.11
	20%	1.0190	0.08	1.0025	23.38
	30%	1.0065	0.10	1.0011	33.10
60	10%	1.0084	0.36	1.0028	254.08
	20%	1.0043	0.51	1.0017	195.16
	30%	1.0023	0.64	1.0012	203.47
90	10%	1.0052	1.20	1.0025	492.28
	20%	1.0023	1.64	1.0009	425.70
	30%	1.0011	2.06	1.0005	451.60

for the tabu procedure, the ratio between the cost of the best solution found with the seven variants and the lower bound and its CPU time is reported.

To study the impact of the input parameters, we have generated a new set of instances. Table 2 shows the combined effect of $|V|$ and link density for given values of $|A|/|E|$, $|A_R|/|A|$, $|E_R|/|E|$ and p. These values have been fixed to 1.5, 0.5, 0.5 and 6, respectively. Density, d, has been computed as $(|A| + |E|)/(|V||V - 1|)$. There is a total of 90 instances: 10 instances for each value of $|V| = 30$, 60, 90 and for each density = 10, 20 and 30%. Note that a density value of 30% in a graph with 6 R-connected components implies a density within each component about 95%. With the given ratio for required links, bigger values for density would lead, in most cases, to instances with only one R-connected component and therefore we would have MCPP instances instead of MRPP instances.

Table 2 shows that, for each combination of $|V|$ and d considered, both algorithms produce high-quality results: the average percentage deviation from the lower bound is always less than 2.3% for the constructive method and less than 0.3% for the tabu algorithm (Mixed 2 variant). Although the UB/LB values are very similar for each procedure, it seems from Table 2 that as density increases, the average solution quality also increases.

Finally, to study separately the effects of p, $|A|/|E|$, $|E_R|/|E|$ and $|A_R|/|A|$, we have generated 80 new instances with $|V| = 60$ and $d = 20\%$. Values have been chosen in such a way that row 5 in Table 2 is shared by Tables 3–6.

Table 3 shows the average results over 10 instances generated for each value of $p = 3$, 6, 9 and fixed values of $|A|/|E| = 1.5$, $|E_R|/|E| = 0.5$ and $|A_R|/|A| = 0.5$. It is known that the complexity of the problem increases with the number p of R connected components. Computational results in Table 3 show, as was expected, that the quality of the solution decreases when p increases.

Table 4 shows the average results over 10 instances generated for each value of $|A|/|E| = 1$, 1.5, 2 and fixed values of $p = 6$, $|E_R|/|E| = 0.5$ and $|A_R|/|A| = 0.5$. Similarly, the average results over 10 instances generated for each value of $|E_R|/|E| = 0.25$, 0.5, 0.75 and fixed values of $|A|/|E| = 1.5$, $p = 6$ and $|A_R|/|A| = 0.5$ are reported in Table 5. In both cases, no significant differences are produced.

Table 3

p	Constructive		Tabu	
	UB/LB	*Seconds*	*UB/LB*	*Seconds*
3	1.0036	0.55	1.0012	42.05
6	1.0043	0.51	1.0017	195.16
9	1.0060	0.47	1.0023	308.86

Table 4

$\|A\|/\|E\|$	Constructive		Tabu	
	UB/LB	*Seconds*	*UB/LB*	*Seconds*
1	1.0043	0.52	1.0014	161.40
1.5	1.0043	0.51	1.0017	195.16
2	1.0030	0.47	1.0013	238.27

Table 5

$\|E_R\|/\|E\|$	Constructive		Tabu	
	UB/LB	*Seconds*	*UB/LB*	*Seconds*
0.25	1.0040	0.53	1.0017	442.01
0.5	1.0043	0.51	1.0017	195.16
0.75	1.0034	0.46	1.0010	149.96

Table 6

$\|A_R\|/\|A\|$	Constructive		Tabu	
	UB/LB	*Seconds*	*UB/LB*	*Seconds*
0.25	1.0063	0.45	1.0023	229.66
0.5	1.0043	0.51	1.0017	195.16
0.75	1.0037	0.47	1.0009	102.37

Finally, Table 6 reports the average results over 10 instances generated for each value of $|A_R|/|A| = 0.25$, 0.5, 0.75 and fixed values of $p = 6$, $|A|/|E| = 1.5$ and $|E_R|/|E| = 0.5$. In this case, it can be noted that as the ratio $|A_R|/|A|$ increases, so does the solution quality in both procedures. This result can be partially explained by the fact that the simplification procedure described in Section 2 performs remarkably well for large values of $|A_R|$, and stages 1 and 2 of the constructive procedure (and then the tabu algorithm) take advantage of this "extra" information.

7. Conclusions

We have developed a first heuristic procedure, based on the resolution of some minimum cost flow and matching problems, that obtains, in short computation time, good solutions for the Mixed Rural Postman Problem. Moreover, we have also presented a local search procedure based on the Tabu Search methodology, to provide high quality solutions.

Experiments with 270 randomly generated graphs were performed to assess the merits of the procedures developed here. Our Tabu Search implementation was shown to perform remarkably well considering the average deviation with respect to a lower bound, obtained with an LP-based cutting plane procedure, and its running time.

Acknowledgements

The authors would like to thank an anonymous referee for his/her helpful comments and suggestions that have greatly improved the contents and presentation of this paper.

Appendix A

Table 7

	P	$\|V\|$	$\|E\|$	$\|E_R\|$	$\|A\|$	$\|A_R\|$		P	$\|V\|$	$\|E\|$	$\|E_R\|$	$\|A\|$	$\|A_R\|$
g3c1	3	20	50	10	80	30	g8c1	8	30	70	20	120	30
g3c2	3	30	140	40	200	50	g8c2	8	40	90	20	190	90
g3c3	3	50	90	60	250	150	g8c3	7	50	50	30	360	60
g3c4	3	50	140	70	190	100	g8c4	8	50	120	70	220	120
g3c5	3	60	140	90	240	80	g8c5	8	60	120	40	180	100
g3c6	3	70	180	100	120	50	g8c6	8	70	150	90	230	70
g3c7	3	80	160	100	300	100	g8c7	6	80	160	50	240	150
g3c8	5	90	130	40	200	50	g8c8	8	90	220	100	160	90
g3c9	3	100	220	150	300	200	g8c9	7	100	150	60	230	100
g3c10	3	100	200	130	250	150	g8c10	7	100	190	120	300	150
g4c1	4	20	105	25	80	50	g9c1	8	30	45	15	95	15
g4c2	3	40	140	20	150	100	g9c2	9	40	80	30	210	60
g4c3	4	50	60	40	250	150	g9c3	9	50	90	60	270	70
g4c4	4	50	110	80	250	100	g9c4	9	50	80	40	150	50
g4c5	4	60	100	50	220	120	g9c5	9	60	150	50	270	150
g4c6	5	70	100	30	200	50	g9c6	8	70	150	100	250	150
g4c7	4	80	190	90	350	50	g9c7	8	80	210	120	200	50
g4c8	3	90	150	100	320	200	g9c8	7	90	150	50	200	120
g4c9	15	100	150	50	340	40	g9c9	7	100	130	70	290	200
g4c10	2	100	150	100	220	130	g9c10	7	100	150	100	240	70
g5c1	5	20	50	20	80	10	g10c1	10	30	20	10	180	30
g5c2	5	40	90	50	150	60	g10c2	10	40	40	20	170	80

Continued

Table 7 (*continued*)

	P	$\|V\|$	$\|E\|$	$\|E_R\|$	$\|A\|$	$\|A_R\|$		P	$\|V\|$	$\|E\|$	$\|E_R\|$	$\|A\|$	$\|A_R\|$
g5c3	5	50	180	30	250	100	g10c3	10	50	80	50	290	40
g5c4	4	50	140	40	210	60	g10c4	10	50	80	40	150	50
g5c5	6	60	110	30	250	50	g10c5	10	60	160	40	280	200
g5c6	5	70	160	100	270	200	g10c6	10	70	160	80	300	130
g5c7	5	80	170	90	190	100	g10c7	10	80	190	130	270	150
g5c8	5	90	180	80	300	150	g10c8	9	90	150	100	250	160
g5c9	4	100	130	60	350	200	g10c9	8	100	160	70	210	100
g5c10	4	100	110	60	250	150	g10c10	10	100	210	150	200	70
g6c1	6	20	60	10	105	15	g11c1	3	30	15	5	105	5
g6c2	6	40	100	30	180	80	g11c2	9	40	35	15	220	20
g6c3	5	50	110	20	270	70	g11c3	12	50	210	10	80	30
g6c4	6	50	160	40	170	100	g10c4	10	50	80	40	150	50
g6c5	6	60	110	80	270	150	g11c5	11	60	150	50	210	120
g6c6	6	70	130	30	270	70	g11c6	11	70	170	90	240	150
g6c7	6	80	160	90	170	50	g11c7	11	80	170	100	250	150
g6c8	5	90	150	100	170	70	g11c8	10	90	170	50	240	150
g6c9	2	100	140	50	230	150	g11c9	10	100	180	120	190	70
g6c10	6	100	180	120	220	130	g11c10	10	100	170	70	220	100
g7c1	6	20	55	5	130	10	g12c1	7	30	105	5	55	5
g7c2	6	40	140	20	100	50	g12c2	11	40	60	10	160	10
g7c3	8	50	140	50	280	30	g12c3	15	50	25	15	320	20
g7c4	7	50	150	90	170	100	g12c4	12	50	160	70	190	110
g7c5	7	60	170	40	240	150	g12c5	12	60	160	40	170	70
g7c6	7	70	150	50	300	200	g12c6	12	70	180	80	210	120
g7c7	7	80	170	100	210	130	g12c7	12	80	180	110	220	90
g7c8	6	90	170	80	270	150	g12c8	11	90	170	60	250	150
g7c9	8	100	150	100	260	60	g12c9	11	100	190	120	170	100
g7c10	6	100	200	120	220	150	g12c10	10	100	190	70	270	150

Table 8

	Constructive		Tabu			Constructive		Tabu	
	UB/LB	*Seconds*	*UB/LB*	*Seconds*		*UB/LB*	*Seconds*	*UB/LB*	*Seconds*
g3c1	1.009	0	1.000	21	g8c1	1.005	0.11	1.000	52
g3c2	1.004	0.11	1.000	7	g8c2	1.009	0.11	1.000	119
g3c3	1.002	0.22	1.001	15	g8c3	1.006	0.21	1.003	223
g3c4	1.004	0.22	1.001	60	g8c4	1.011	0.22	1.002	164
g3c5	1.011	0.39	1.001	75	g8c5	1.019	0.33	1.002	341
g3c6	1.009	0.5	1.005	15	g8c6	1.015	0.5	1.005	436
g3c7	1.014	0.77	1.003	56	g8c7	1.003	0.66	1.001	363
g3c8	1.014	0.61	1.001	274	g8c8	1.013	0.88	1.002	646
g3c9	1.004	1.21	1.002	223	g8c9	1.007	1.05	1.000	569
g3c10	1.012	1.2	1.006	103	g8c10	1.013	1.09	1.004	476

Continued

Table 8 (*continued*)

	Constructive		Tabu			Constructive		Tabu	
	UB/LB	*Seconds*	*UB/LB*	*Seconds*		*UB/LB*	*Seconds*	*UB/LB*	*Seconds*
g4c1	1.000	0.06	1.000	2	g9c1	1.099	0.05	1.000	16
g4c2	1.001	0.11	1.000	15	g9c2	1.004	0.11	1.000	227
g4c3	1.006	0.22	1.000	83	g9c3	1.012	0.27	1.001	309
g4c4	1.006	0.22	1.000	77	g9c4	1.011	0.16	1.000	45
g4c5	1.009	0.22	1.002	146	g9c5	1.008	0.33	1.001	316
g4c6	1.002	0.39	1.001	176	g9c6	1.020	0.49	1.007	59
g4c7	1.008	0.71	1.006	48	g9c7	1.016	0.66	1.005	426
g4c8	1.005	0.82	1.002	69	g9c8	1.007	0.77	1.003	394
g4c9	1.071	1.15	1.013	589	g9c9	1.006	0.77	1.003	580
g4c10	1.009	0.99	1.005	135	g9c10	1.024	0.99	1.012	513
g5c1	1.022	0.05	1.000	6	g10c1	1.085	0.05	1.011	36
g5c2	1.007	0.11	1.000	49	g10c2	1.028	0.11	1.000	250
g5c3	1.012	0.22	1.000	130	g10c3	1.025	0.27	1.005	93
g5c4	1.002	0.22	1.000	95	g10c4	1.034	0.17	1.000	40
g5c5	1.018	0.27	1.000	213	g10c5	1.015	0.33	1.001	351
g5c6	1.004	0.49	1.001	237	g10c6	1.026	0.49	1.006	372
g5c7	1.007	0.66	1.002	261	g10c7	1.020	0.61	1.006	388
g5c8	1.001	0.93	1.000	390	g10c8	1.006	0.77	1.000	471
g5c9	1.007	0.99	1.003	321	g10c9	1.021	0.93	1.009	630
g5c10	1.003	0.88	1.001	283	g10c10	1.035	1.04	1.006	744
g6c1	1.009	0.06	1.000	42	g11c1	1.000	0	1.000	4
g6c2	1.010	0.11	1.000	95	g11c2	1.020	0.11	1.004	12
g6c3	1.010	0.22	1.002	121	g11c3	1.072	0.17	1.000	442
g6c4	1.010	0.16	1.000	201	g10c4	1.011	0.22	1.001	182
g6c5	1.005	0.33	1.001	183	g11c5	1.019	0.39	1.003	567
g6c6	1.003	0.44	1.002	249	g11c6	1.027	0.49	1.006	246
g6c7	1.023	0.61	1.004	264	g11c7	1.010	0.6	1.003	445
g6c8	1.020	0.72	1.002	284	g11c8	1.005	0.77	1.003	538
g6c9	1.000	0.94	1.000	105	g11c9	1.028	0.94	1.013	597
g6c10	1.006	0.99	1.003	1038	g11c10	1.019	0.88	1.002	869
g7c1	1.038	0.06	1.004	31	g12c1	1.031	0	1.000	2
g7c2	1.018	0.17	1.000	117	g12c2	1.208	0.05	1.000	72
g7c3	1.017	0.27	1.002	423	g12c3	1.158	0.17	1.025	293
g7c4	1.014	0.22	1.002	74	g12c4	1.023	0.27	1.005	234
g7c5	1.008	0.39	1.001	217	g12c5	1.011	0.27	1.004	382
g7c6	1.004	0.49	1.000	353	g12c6	1.018	0.5	1.008	410
g7c7	1.015	0.66	1.003	367	g12c7	1.034	0.61	1.010	484
g7c8	1.006	0.87	1.002	431	g12c8	1.023	0.77	1.001	637
g7c9	1.019	0.99	1.006	630	g12c9	1.020	0.88	1.006	457
g7c10	1.005	1.04	1.001	424	g12c10	1.013	1.04	1.002	151

References

[1] Edmonds J, Johnson EL. Matching, Euler tours and the Chinese postman problem. Mathematical Programming 1973;5:88–124.
[2] Papadimitriou CH. On the complexity of edge traversing. Journal of ACM 1976;23:544–54.
[3] Frederickson GN. Approximation algorithms for some postman problems. Journal of ACM 1979;26:538–54.
[4] Pearn WL, Liu CM. Algorithms for the Chinese postman problem on mixed networks. Computers & Operations Research 1995;22:479–89.
[5] Grötschel M, Win Z. A cutting plane algorithm for the Windy postman problem. Mathematical Programming 1992;55:339–58.
[6] Nobert Y, Picard JC. An optimal algorithm for the mixed Chinese postman problem. Networks 1996;27:95–108.
[7] Lenstra JK, Rinnooy Kan AHG. On general routing problems. Networks 1976;6:273–80.
[8] Pearn WL, Wu TC. Algorithms for the rural postman problem. Computers & Operations Research 1995;22:819–28.
[9] Hertz A, Laporte G, Nanchen P. Improvement procedures for the undirected rural postman problem. INFORMS Journal on Computing, in press.
[10] Corberán A, Sanchis JM. A polyhedral approach to the rural postman problem. European Journal of Operational Research 1994;79:95–114.
[11] Corberán A, Sanchis JM. The general routing problem polyhedron: facets from RPP and GTSP polyhedra. European Journal of Operational Research 1998;108:538–50.
[12] Letchford AN. New inequalities for the general routing problem. European Journal of Operational Research 1997;96:317–22.
[13] Eiselt HA, Gendreau M, Laporte G. Arc routing problems, Part I: the Chinese postman problem. Operations Research 1995;43:231–42.
[14] Eiselt HA, Gendreau M, Laporte G. Arc routing problems, Part II: the rural postman problem. Operations Research 1995;43:399–414.
[15] Assad AA, Golden BL. Arc routing methods and applications. In: Ball M et al., editors. Network routing. Handbooks in OR and MS, vol. 8. Amsterdam: North-Holland, 1995.
[16] Alvarez-Valdés R, Benavent E, Campos V, Corberán A, Mota E, Tamarit JM, Valls V. ARC, a computerised system for urban garbage collection. TOP (Journal of the Spanish OR Society) 1993;1:89–106.
[17] Christofides N, Benavent E, Campos V, Corberán A, Mota E. An optimal method for the mixed postman problem. In: Thoft-Christensen P, editor. System Modelling and Optimization, Lecture Notes in Control and Information Sciences, vol. 59. Berlin: Springer, 1984.
[18] Christofides N, Campos V, Corberán A, Mota E. An algorithm for the rural postman problem on a directed graph. Mathematical Programming Study 1986;26:155–66.
[19] Ford LR, Fulkerson DR. Flows in networks. Princeton, NJ: Princeton University Press, 1962.
[20] Glover F, Laguna M. Tabu search. Dordrecht: Kluwer Academic, 1997.
[21] Corberán A, Romero A, Sanchis JM. Facets of the mixed rural postman problem polyhedron. Communication presented at the EURO/INFORMS Conference. Barcelona, July 1997.
[22] Padberg MW, Rao MR. Odd minimum cut-sets and b-matchings. Mathematics and Operations Research 1982;7:67–80.

Angel Corberán is a Titular Professor in the Department of Statistics and Operations Research at the University of Valencia in Spain. His areas of research include polyhedral combinatorics, routing problems, production planning and the development of heuristic procedures in combinatorial optimization. He is Associate Editor of TOP (Journal of the Spanish OR Society).

Rafael Martí is a Titular Professor in the Department of Statistics and Operations Research at the University of Valencia in Spain. His areas of research include graph drawing, routing problems and metaheuristics in combinatorial optimization.

Antonio Romero is a Post-doctoral Student in the Department of Statistics and Operations Research at the University of Valencia in Spain. He received his doctoral degree in Mathematics from the University of Valencia. He is currently working on routing problems.